Hashing

To access an element stored in an array, we need to know the index number. If we have the value, we have to search to find the index/position of the element. In a list of nodes, in any case a search must be done. Same as in a tree, although in a tree it may be faster.

But it would be nice if we could access the data based on a key (that can be the data itself or something else). And it would be nice if it were fast. And that's what **Hashing** is about. It's very common to have structures like Python dicts, where we access the element based on some key. We don't need to know it's index beforehand.

This can be achieved with hashing. The first thing we need is a **hashing function**, that takes some input and generates some numeric value based on that input.
Hashing functions should have certain properties:

- They are expected to work very quickly generating their numeric output. They can take a little more or less time, but they have to be of time complexity $O(1)$.
- They are also expected to always generate the same output given the same key. This is important.

- The range of numbers to be generated should be limited. The inputs are usually unlimited, but output range is generally relatively small.
- The outputs should be well distributed in the output range. All possible outputs should have equal opportunity of being generated.

The hashing function is a mathematical problem that we are not going to cover here, but there are known solutions that give a good approximation to what is expected from hashing functions.

A very rough example of hashing function is one that get us a numeric value based on the length of a text passed as a parameter. With that in mind, data with the key "Hello" would be stored in position 5 and with the key "Friends" would be stored in position 7. To limit the range of the output, we could use the Python "modulo" expression, represented with the `%` character. So our hashing function would be something like: `len(input_text) % 10`. With this, the input "Hello world" outputs a 1.

0	1	2	3	4	5	6	7	8	9
	Hello world				Hello		Friends		

One evident problem is that of course there are many texts with the same lengths or, in other words, many inputs that are going to give the same hashing output. This is what is called **Collision**. There are always going to be two or more keys that give the same hashing result, given that the possible number of inputs to the functions is usually much bigger than the number of outputs.

Hashing algorithms try to minimize collisions generating outputs with similar probability of being generated. That minimizes the possibility of collisions. But all in all, collisions can't be avoided. Instead, a few methods have been developed to deal with them. And we will try to see a little bit of them shortly.

Hash tables

The **hash table** is the table that works with a hashing function to store values/keys in it. This table can actually be implemented with arrays or lists, and we could use the IntArray structure we created in chapter 2. But for the sake of simplicity and due to IntArray only be able to store integers, we are going to use a Python list.

We start implementing a Hash table by creating the class that store the values, like the node in previous structures:

```
class HashItem():
    def __init__(self, key, value):
        self.key = key
        self.value = value

    def __repr__(self):
        return f'{{{self.key}: {self.value}}}'
```

And then we can start creating a hash table:

```
class HashTable():
    def __init__(self, size=20):
        self.size = size
        self.slots = [None] * size
        self.used_slots = 0

    def __repr__(self):
        text = ''
        for index, slot in enumerate(self.slots):
            if slot:
                text += f', {index}: {slot}'
        plural = '' if self.used_slots == 1 else 's'
        return f'<HashTable ({self.used_slots} element{plural}): [{text.lstrip(", ")}]'
```

The `__repr__()` methods in the classes are a convenience to help debugging code in the REPL.

A couple of things to note. The length of the array can be passed as a parameter to the class constructor, and here we have arbitrarily decided to give it a default value of 20 elements. As we are using a Python list, we can always have its length with the `len` function, but we have made it explicit with its own `size` variable. And finally, there is a `used_slots` variable that count how many slots are used. This way we have a quick way to know if the hash table is full or near full.

And then we need a hashing function. We are going to use the same example as before, even if it is quite bad for a hashing function. But it's a function easy to understand and it's also easy to create collisions for demo purposes:

```
    def _hash(self, key):
        return len(key) % self.size
```

Notice the use of the modulo expression to limit the range of the outputs. Another important thing to note is that our hashing function only accepts strings or actually anything that accepts the `len` function, but the coder has to decide what can be used as a key to the hashing function and how it is managed.

We also would like to add an insert function, but before we do that, we have to talk a little bit about collisions and how to handle them.

Collisions

As said before, collisions can't be avoided. We just need some mechanism to deal with them. There are different options...

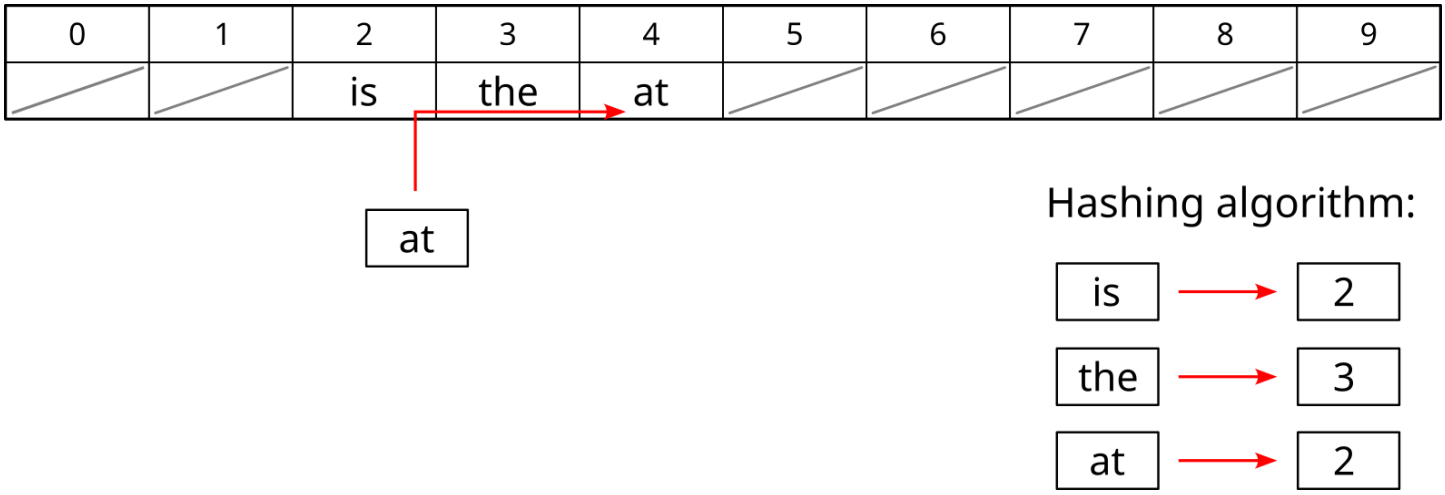
Open addressing

Open addressing uses the same hash table to solve the collisions. If there is a collision, then another slot is used. But which one?

- In **linear probing**, if the slot that the key should have used has index q, then the slots that are at a fixed interval p are *probed*, that is, the slot at $q + p$ it's checked to see if it is in use or not. If that slot is also in use, the slot at $q + 2 \cdot p$ is probed, and then the one at $q + 3 \cdot p$, and so on... Usually $p = 1$, so that means that the slots probed are $q + 1$, then $q + 2$, then $q + 3$, etc...
- In **quadratic probing**, the expression to find the next slot to probe is quadratic. For example, $q + p^2$
- In **double hashing**, the next slot to be probed is calculated with a secondary hashing function.

There are more methods, but these ones are the most common ones.

Let's see an example:



Notice

The image shows only the keys of the elements, but also some values are stored besides the keys. Review the `HashItem` class if necessary.

With linear probing, the "at" text is pushed until the fourth slot, as the second and third slots are in use. Using open addressing means that all functions should be aware how things work. That means, that, for example, to read the value of the key from the table, we start searching from the slot the key originally should go, but if it is not found, then the next slots have to be also searched.

To give an example, the hashing function output for "at" is 2. But if we check that position, we can see that "at" key is not there. We have to continue probing the next positions until we find the key, or an empty slot is found. If an empty slot is found, the key is not in the table.

The code for a method that tries to find the key would be something like this:

```python
    def _find_key(self, start, key):
        """
        Starting from 'start' try to find 'key'.

        Parameters:
        - 'start': Starting index
        - 'key': The key to be found

        Returns: The index position of the key or None if not found
        """
        # Start to search from the given position
        current = start

        # While current position is occupied and it doesn't contain the key, enter the loop
        while self.slots[current] and self.slots[current].key != key:
            # Increment current, but if the end or the table is reached,
            # continue from the start of the table.
            current = (current + 1) % self.size
            # If we reach again the given position, that means
            # a whole cycle has been completed and the key was not found
            if current == start:
                return None

        # After the loop, current points to a free position or
        # to the position with the key
        if self.slots[current]:
            return current
        else:
            return None
```

We are going to need a similar function to find a free slot as the insertion procedure is going to need it. If the slot where a key should go is occupied, then start looking in the next positions until a free slot is found. The function is almost the same as the previous one. So, it's your turn to give it a try!

Exercise!

Open the Moodle environment and write a method that from a given position finds the next free slot of the table.

Don't continue until you have finished the exercise!

You should have came up with something like this:

```python
    def _find_free_slot(self, start):
        """
        Starting from 'start' find the next free slot available.

        Parameters:
        - 'start': Starting point for the search.

        Returns: The index of the next free slot or None if no free slots
        """
        # Start to search from the given position
        current = start

        # While that position is in use, enter the loop
        while self.slots[current]:
            # Increment current, but if the end or the table is reached,
            # continue from the start of the table.
            current = (current + 1) % self.size

            # If we reach again the given position, that means
            # a whole cycle has been completed and there was no free
            # positions available
            if current == start:
                return None

        # After the loop, current points to a free position
        return current
```

Once the free slot has been found, the insertion is quite easy. Our function is going to support insertion and updating. If the key is already in the table, its value should be updated with the new value. Because of how method works, we are going to call it `put` instead of `insert`. But other than that, it shouldn't be difficult to implement, so this is another one for you!

Exercise!

Open the Moodle environment and write a method that add or updates a key with a value.

Don't continue until you have finished the exercise!

The method should be like this:

```
    def put(self, key, value):
        """
        Add or updates a key with a value in the hash table

        Parameters:
        - 'key': The key to add or update.
        - 'value': The value of the key

        Returns: None
        """
        # Raise an error if no space available
        if self.used_slots == self.size:
            raise(MemoryError('Hash table is full'))

        #  Try to find the key
        h = self._hash(key)
        q = self._find_key(h, key)
        if q is not None:
            # If found, update the value
            self.slots[q].value = value
        else:
            # If not found, find a free slot
            q = self._find_free_slot(h)
            # Add a new element.
            self.slots[q] = HashItem(key, value)
            # And increase used_slots
            self.used_slots += 1
```

Another one you can try is a `get` method. We can try to mimic the Python dict's `get` method. This method accepts a key as parameter and returns the value if the key is found. If not found, function returns None by default, but the method also accepts a second parameter as the value to return if the key is not found. In our case, the parameter name is going to be `alternative`.

Exercise!

Open the Moodle environment and write a method that returns the value of a key or a value given as a second parameter if the key is not found. You can use the helper methods to help you in the task.

Don't continue until you have finished the exercise!

In this case, we should end up with something like this:

```
    def get(self, key, alternative=None):
        """
        Get the value of 'key'.

        Parameters:
        - 'key': The key to find
        - 'alternative': An alternative value if key is not found.
                         Optional. Default value=None

        Returns: The value of 'key' or alternative value
        """
        # Try to find the key
        h = self._hash(key)
        q = self._find_key(h, key)
        if q is None:
            # If key was not found return None
            return alternative
        else:
            # If key was found, return the value of the key
            return self.slots[q].value
```

A trickier problem is the deletion of keys when using open addressing, because the deletion of a key can affect other keys as there is now a free slot that could end some lookups. One possible solution is the use of a "deleted" property in `HashItem`, that marks a slot as free without actually nullifying it. Other possible solution, at least on linear probing, is to move some elements to fill the gap that showed up after deleting an element.

A possible implementation could be this:

```python
    def remove(self, key):
        """
        Remove a key from the table

        Parameters:
        - 'key': key to remove

        Returns: None
        """
        # Find the position of given key
        freed_pos = self._find_key(self._hash(key), key)
        # If not found, return
        if freed_pos is None:
            return None
        # Define a helper lambda that gives the separation
        # in positions of a given position and the original
        # position a key should go.
        separation = lambda pos, elem: (pos - self._hash(elem.key)) % self.size
        # Decrement the number of used slots
        self.used_slots -= 1

        # Loop
        while True:
            # Empty the position
            self.slots[freed_pos] = None
            # Prepare a helper pointer
            candidate_pos = freed_pos
            while True:
                # Move the helper pointer 1 position
                candidate_pos = (candidate_pos+1) % self.size
                # Save that positions content
                current_elem = self.slots[candidate_pos]
                # If position is empty, function can return
                if current_elem is None:
                    return
                # If the separation between current element's original position
                # and the position that was emptied is less than the separation
                # between the element's original position and the helper position
                # then exit the loop (to move the element to the emptied position)
                if separation(freed_pos, current_elem) < separation(candidate_pos,
current_elem):
                    break
            # Move the element to the emptied position
            self.slots[freed_pos] = current_elem
            # New empty position is the helper position
            freed_pos = candidate_pos
```

Open addressing possible problems are, for example, that linear probing and quadratic probing are prone to different types of **clustering**. Clustering occurs when elements cluster or group together in large batches of occupied slots.

Other possible problem arises when the **load factor**, i.e. how much of the table is occupied, is too high. Load factor goes from 0 to 1 (0% to 100%) and many times performance starts degrading rapidly already at 0.75 (75%), as for every action it takes more time to locate elements or free space. The solution to this is usually to move the data to a larger table (for example to a table double the size of the original), so the load factor lowers a little bit.

Chaining

Chaining is another strategy to deal with collisions. The idea is quite simple: instead of single elements, each position of the table contains a list of elements. The functions only have to deal with the list in question.

0	1	2	3	4	5	6	7	8	9
		is	the	bell		corner		calendar	
		at	boy	tree		detail		flexible	
			bus					interval	
			and						

Chaining methods are simpler than their Open addressing counterparts, as some checks and procedures can be skipped. Free slots don't need to be found for example.

The downside of this method is that it needs more memory than Open addressing. If some of the lists grow too much, then it can affect the performance of the hash table, as lookups are going to take more time for certain keys. To avoid this problem, it could be possible to use BSTs instead of lists. The potential problem could be unbalancing trees. So probably a good idea to use self-balancing trees.

On the other hand, with chaining the table never becomes full.

Using brackets

To use the hash table like Python dicts, using the brackets characters `[]`, the Python special methods `__getitem__()` and `__setitem__()` have to be used. Fortunately, the actions are already covered by some methods already implemented, so we only need to call them:

```
    def __setitem__(self, key, value):
        """
        Set or updates the value of element 'key'.

        Use: hashtable[key] = value

        Parameters:
        - 'key': The key to add or update
        - 'value': The value of the key

        Returns: None
        """
        self.put(key, value)
```

```
    def __getitem__(self, key):
        """
        Get value of element 'key'

        Use: hashtable[key]

        Parameters:
        - 'key': The key to get the value

        Returns: The value of given key. Raises error if key was not found
        """
        if result := self.get(key):
            return result
        else:
            raise(KeyError('Key not found'))
```

The `__getitem__` method works like the Python dict do, raising an error if the key is not found.

Summary

Hash tables are used in different kind of scenarios. The most evident one, could be in a Dict type structure, where obviously the elements have to be accessed by a key and not by an index.

Another use case are the symbol tables interpreters or compilers of programming languages use to maintain track of the identifiers (variables, functions, classes, etc...) that have been declared in the program. The identifiers themselves are the keys to the hash table (Symbol tables) and the information related to the identifier is the value: type of identifier, actual value, etc...

It's easy to maintain, as most programming languages do, different symbol tables that act like scopes for the identifiers.

The performance analysis of these algorithms depends on the implementation. It's not the same to implement the hash table with arrays or with lists, nor is use Open addressing for collision handling or Chaining. The general idea anyway is that the lookup of a key should be of time complexity $O(1)$. Additions should also be of time complexity $O(1)$, but as we have seen, the load factor of the hash table (how full it is) can affect here. If the hash table is near full, the content should be moved to a new bigger table, and this process is of time complexity $O(n)$, as all the elements should be transferred.

The hashing function is important. We have used a silly example of hashing function, as it was easy to create collisions with it and test the algorithms. A better (but not much), but still quite simple function could have been this:

```
    def _hash(self, key):
        """
        Hashing function. Can be changed for a custom one.
        """
        return sum((index+1) * ord(char) for index, char in enumerate(key)) % self.size
```